FitSync Backend

شرح كامل لخدمة تتبّع الوجبات (Meal Tracking API)

Laravel 12 • Laravel Sanctum • MySQL • PHPUnit

مرجع للمناقشة: ماذا بنينا، كيف يعمل الكود، كيف تُشغَّل الخدمة، وما الأسئلة المتوقعة

تاريخ الإعداد: 7 تشرين الثاني 2026

المحتويات

1. فكرة الخدمة والأدوار
2. التقنيات وبنية المشروع
3. قاعدة البيانات
4. المصادقة (Sanctum)
5. رحلة الطلب داخل Laravel
6. واجهات اللاعب (Athlete)
7. واجهات الأخصائي (Nutritionist)
8. الوقت والمناطق الزمنية
9. الأمان وعزل البيانات
10. ماذا وجدنا وماذا أصلحنا
11. الاختبارات و Postman
12. تشغيل المشروع وسيناريو العرض
13. أسئلة متوقعة وأجوبتها
14. حدود الخدمة وتطويرات مستقبلية

1. فكرة الخدمة والأدوار

FitSync تطبيق لياقة وتغذية. الجزء الذي نفّذناه هو **Meal Tracking**: اللاعب يسجّل ما أكله خلال اليوم، والنظام يقارن مجموع السعرات بالهدف اليومي الذي حدّده له أخصائي التغذية، والأخصائي يبني الخطط الغذائية ويتابع سجلات لاعبيه.

الدور	ماذا يستطيع أن يفعل
Athlete (لاعب)	تسجيل وجبة، تعديلها، حذفها، عرض وجبات يوم معيّن، ملخص السعرات (المسجَّل / الهدف / المتبقي / الزيادة)، عرض الوجبات المخططة من خطته.
Nutritionist (أخصائي)	إنشاء خطط غذائية للاعبين (هدف سعرات + فترة زمنية)، إضافة وجبات مخططة للخطة وتعديلها وحذفها، عرض قائمة لاعبيه، ومشاهدة سجلات وجبات لاعب لديه خطة فعّالة.

المفاهيم الأساسية

- **Meal Log (سجل وجبة):** ما أكله اللاعب فعلاً (اسم، نوع، سعرات، وقت الأكل، ملاحظات). يخص لاعباً واحداً.
- **Meal (وجبة مخططة):** وجبة مقترحة داخل خطة الأخصائي. اللاعب يستطيع ربط سجله بوجبة مخططة عبر `meal_id` (اختياري).
- **Nutrition Plan (خطة غذائية):** تربط أخصائياً بلاعب، وفيها `daily_calories_target` وتاريخ بداية ونهاية وحالة فعّال/غير فعّال.

2. التقنيات وبنية المشروع

المكوّن	الاستخدام
Laravel 12 (PHP 8.2)	إطار العمل. الخدمة API فقط: كل المسارات تحت `/api` وتردّ JSON.
Laravel Sanctum 4	مصادقة بـ Bearer Token (جدول `personal_access_tokens`).
MySQL	قاعدة البيانات `fitsync`. الاختبارات تعمل على SQLite داخل الذاكرة.
PHPUnit	39 اختباراً آلياً.
Postman	مجموعة طلبات جاهزة مع اختبارات `pm.test`.

الملفات المهمة

```
app/
 ├─ Models/            User, Athlete, Nutritionist, NutritionPlan, Meal, MealLog
 ├─ Http/
 │   ├─ Controllers/Api/
 │   │    ├─ AuthController.php         login / logout
 │   │    ├─ MealLogController.php      عرض الأخصائي لسجلات لاعب + واجهات اللاعب
 │   │    └─ NutritionistController.php الخطط والوجبات المخططة وقائمة اللاعبين
 │   ├─ Requests/      MealDayRequest, SaveMealLogRequest,
 │   │                 SaveNutritionPlanRequest, SaveMealRequest   (التحقق + الصلاحية)
 │   └─ Resources/     MealLogResource, MealResource, NutritionPlanResource (شكل JSON)
bootstrap/app.php      تسجيل المسارات + معالجة الأخطاء كـ JSON
routes/api.php         كل المسارات
config/meal_tracking.php  المنطقة الزمنية لحدود اليوم (Asia/Damascus)
database/migrations    الجداول     database/seeders  بيانات تجريبية
tests/Feature          MealTrackingTest, NutritionistPlanTest
postman/               collection + environment
```

3. قاعدة البيانات

```
users (id, name, email*, password, role, is_active)
  | 1 ── 1                        1 ── 1
  ├── athletes (athlete_id PK, user_id*)        ──┐
  └── nutritionists (nutritionist_id PK, user_id*) ─┤
                                                    |
nutrition_plans (nutrition_plan_id PK,              |
   athlete_id FK ──► athletes,                      |
   nutritionist_id FK ──► nutritionists,            |
   title, daily_calories_target, start_date,        |
   end_date, is_active)                             |
      | 1 ── N                                      |
meals (meal_id PK, nutrition_plan_id FK,            |
   name, meal_type, description, calories)          |
      | 1 ── N (اختياري)                           |
meal_logs (meal_log_id PK, athlete_id FK ──► athletes,
   meal_id FK ──► meals NULL, name, meal_type,
   calories, consumed_at, notes)  index(athlete_id, consumed_at)

personal_access_tokens (Sanctum)        * = unique
```

- الدور `role` نص في جدول `users` (`athlete` أو `nutritionist`) مع `is_active` لتعطيل الحساب.
- كل مستخدم له صف في `athletes` **أو** `nutritionists` بحسب دوره. المفاتيح الأساسية مخصصة (`athlete_id` بدل `id`)، لذلك كل علاقة في الـ Models تحدّد اسم المفتاح صراحةً.
- حذف `meal` مرتبط بسجل لا يحذف السجل: `meal_logs.meal_id` يصبح `NULL` (`nullOnDelete`). أما حذف مستخدم/لاعب/أخصائي له بيانات فمرفوض (`restrictOnDelete`) حفاظاً على السجلات.

- الفهرس `(athlete_id, consumed_at)` يسرّع استعلام «وجبات لاعب في يوم معيّن»، وهو أكثر استعلام يُنفَّذ.

4. المصادقة (Sanctum)

تسجيل الدخول `POST /api/login`

1. التحقق أن `email` و `password` موجودان.
2. البحث عن المستخدم والتحقق من كلمة المرور بـ `Hash::check` ومن أن الحساب `is_active`.
3. عند أي فشل (مستخدم غير موجود / كلمة مرور خاطئة / حساب معطّل) تُرجَع **نفس الرسالة** وكود 401، حتى لا يعرف المهاجم أي الإيميلات مسجّلة.
4. عند النجاح: `createToken('postman-demo', ['*'], now()->addHours(8))`؛ التوكن صالح 8 ساعات، ويُخزَّن في قاعدة البيانات **مُشفَّراً بـ hash**، والنص الأصلي يُعرَض مرة واحدة فقط في الرد.
5. الطلب محمي بـ `throttle:5,1` : 5 محاولات في الدقيقة لكل IP (ضد تخمين كلمات المرور).

```
POST /api/login        {"email":"athlete@example.test","password":"DemoPassword!123"}
200 → {"token_type":"Bearer","token":"1|Xk3...","user":{"id":1,"name":"Demo
Athlete","role":"athlete"}}
```

استخدام التوكن

كل المسارات عدا `/login` داخل `Route::middleware('auth:sanctum')`. العميل يرسل الهيدر `Authorization: Bearer <token>`. Sanctum يبحث عن التوكن في الجدول ويحدّد المستخدم، فيصبح متاحاً عبر `$request->user()`.

تسجيل الخروج `POST /api/logout`

يحذف **التوكن الحالي فقط** (`currentAccessToken()->delete()`) ويردّ 204. بعدها أي طلب بنفس التوكن يعطي 401.

> **تعديل أضفناه:** في `bootstrap/app.php` جعلنا كل أخطاء `api/*` تُرجَع JSON، وأوقفنا إعادة التوجيه إلى صفحة login (غير موجودة في API). النتيجة: طلب بلا توكن يعطي `401 {"message":"Unauthenticated."}` حتى لو لم يرسل العميل `Accept: application/json`. قبل ذلك كان يعطي خطأ 500 (`Route [login] not defined`).

5. رحلة الطلب داخل Laravel

مثال: اللاعب يسجّل وجبة `POST /api/meal-logs`.

```
HTTP request
  │
  ▼ routes/api.php            Route::post('/meal-logs') داخل auth:sanctum
  ▼ Middleware                auth:sanctum → من هو المستخدم؟ (401 وإلا JSON)
  ▼ SaveMealLogRequest        authorize(): لاعب فعّال وله صف في athletes(403 وإلا) ؟
  │                           rules():     تحقق من الحقول (وإلا 422 مع تفاصيل كل حقل)
  ▼ MealLogController@store   $athlete->mealLogs()->create($data)
  ▼ MealLog (Model)           $fillable يحدد الحقول المسموحة فقط
  ▼ MealLogResource           يحدد شكل JSON الراجع (calories رقماً، تواريخ ISO)
  ▼ HTTP 201 + JSON
```

لماذا هذا التقسيم؟

- **FormRequest:** التحقق والصلاحية خارج الـ Controller فيبقى نظيفاً.
- **Controller:** منطق الأعمال فقط (حدود اليوم، الملخص، التملّك).
- **Resource:** فصل شكل الرد عن شكل جدول قاعدة البيانات (مثلاً `calories` تُرجَع رقماً `450.5` وليس نصاً `"450.50"`).

6. واجهات اللاعب (Athlete)

كل واجهات اللاعب تبدأ بالدالة `athlete()` في الـ Controller: تتأكد أن المستخدم فعّال، دوره `athlete` ، وله صف في `athletes` ، وإلا ترجع 403. كل الاستعلامات تمر عبر `$athlete->mealLogs()` فلا يمكن لأي استعلام أن يخرج عن سجلات هذا اللاعب.

المسار	الوظيفة والتفاصيل
`POST /meal-logs`	تسجيل وجبة. الحقول: `meal_type` (breakfast/lunch/dinner/snack)، `name` (≤150)، `calories` (0 – 999999.99، منزلتان عشريتان)، `consumed_at` بصيغة `2026-11-07T12:30:00+03:00` ولا يكون في المستقبل، `notes` و `meal_id` اختياريان. يردّ 201.
`GET /meal-logs?date=&per_page=&page=`	وجبات يوم معيّن مرتبة بالوقت، مع ترقيم صفحات (15 افتراضياً، حتى 100). يعيد `data` و `links` و `meta` و `date` و `timezone` .
`PATCH /meal-logs/{id}`	تعديل جزئي: أرسل الحقول المراد تغييرها فقط. سجل لاعب آخر ← 404.
`DELETE /meal-logs/{id}`	حذف سجله فقط ← 204. سجل لاعب آخر ← 404.
`GET /meal-logs/summary?date=`	عدد الوجبات، مجموع السعرات، الهدف اليومي، المتبقي، الزيادة.
`GET /planned-meals?date=`	الخطط الفعّالة التي تغطي هذا اليوم مع وجباتها المخططة.

الملخص اليومي (summary) خطوة بخطوة

1. نحوّل التاريخ إلى حدّي اليوم (بداية ونهاية) بتوقيت دمشق ثم إلى UTC.
2. استعلام واحد: `COUNT(*)` و `SUM(calories)` لسجلات اللاعب ضمن الحدّين.
3. نبحث عن الخطة الفعّالة التي `start_date ≤ date ≤ end_date` . إن وُجدت أكثر من واحدة نردّ **409** (هدف يومي غامض).
4. الهدف = `daily_calories_target` . المتبقي = `max(0, الهدف − المجموع)` ، الزيادة = `max(0, المجموع − الهدف)` . إذا لا توجد خطة تكون القيم `null` .

```
GET /api/meal-logs/summary?date=2026-11-07
200 → {"date":"2026-11-07","timezone":"Asia/Damascus","logged_meals_count":3,
       "logged_calories":1320,"target_calories":2000,
       "remaining_calories":680,"excess_calories":0}
```

(هذا الرد حقيقي من تشغيل الخدمة على البيانات التجريبية: 450 + 650 + 220 = 1320 مقابل هدف 2000.)

مثال تسجيل وجبة

```
POST /api/meal-logs
{"name":"Grilled Chicken","meal_type":"lunch","calories":450.5,
 "consumed_at":"2026-11-07T00:00:00+03:00","notes":"test"}

201 → {"data":{"meal_log_id":4,"athlete_id":1,"meal_id":null,"name":"Grilled Chicken",
        "meal_type":"lunch","calories":450.5,"consumed_at":"2026-11-06T21:00:00+00:00", ...},
       "message":"تم تسجيل الوجبة."}
```

أخطاء التحقق (422)

```
{"message":"The consumed at field must be a date before or equal to now.",
 "errors":{"consumed_at":["..."]}}
```

7. واجهات الأخصائي (Nutritionist) — أُضيفت في هذه المرحلة

كانت الخطط والوجبات المخططة تُنشأ فقط عبر الـ Seeder. أضفنا لها واجهات كاملة في `NutritionistController` حتى يعمل النظام فعلاً بدون تدخّل في قاعدة البيانات.

المسار	الوظيفة
`GET /nutritionist/athletes`	اللاعبون الذين لهم خطة عند هذا الأخصائي (id، الاسم، ال `athlete_id`.
`GET /nutritionist/athletes/{id}/meal-logs?date=`	سجلات لاعب ليوم معيّن، بشرط وجود خطة فعّالة بينهما ؛ الحالي».
`GET /nutritionist/plans`	خططي (ترقيم صفحات، فلتر `athlete_id`).
`POST /nutritionist/plans`	إنشاء خطة: `ily_calories_target, start_date, end_date` (+ `is_active`).
`GET/PATCH/DELETE /nutritionist/plans/{id}`	عرض (مع الوجبات) / تعديل جزئي / حذف. الحذف يحذف و `DB::transaction` ، وسجلات اللاعب تبقى (`meal_id`
`POST /nutritionist/plans/{planId}/meals`	إضافة وجبة مخططة للخطة.
`PATCH/DELETE /nutritionist/meals/{id}`	تعديل / حذف وجبة مخططة.

قواعد مهمة طبّقناها

- **منع تداخل الخطط الفعّالة لنفس اللاعب:** الدالة `assertNoOverlap` ترفض (422) أي خطة فعّالة تتقاطع فترتها مع خطة فعّالة أخرى للاعب نفسه: `start_a ≤ end_b` و `end_a ≥ start_b` . السبب: الملخص

اليومي يحتاج هدفاً واحداً فقط (وإلا 409). الخطة غير الفعّالة مسموح لها بالتداخل.

- **التملّك:** الأخصائي لا يرى ولا يعدّل إلا خططه ووجباتها. خطة أخصائي آخر تظهر له 404 كأنها غير موجودة.
- **تغيير اللاعب ممنوع بعد الإنشاء:** `athlete_id` محظور في PATCH. وكذلك `nutritionist_id` و `nutrition_plan_id` محظوران دائماً في الجسم (يؤخذان من المستخدم المسجَّل ومن الـ URL).
- **تحقق من التواريخ في التعديل:** إذا عدّل الأخصائي `end_date` فقط، نقارنه مع `start_date` المخزَّن (في الـ Controller لأن الـ FormRequest لا يرى القيمة القديمة).
- **توحيد رد 403:** عرض سجلات لاعب لا يخص الأخصائي، أو لاعب غير موجود أصلاً، يعطي نفس 403 — حتى لا يستطيع تخمين أي أرقام لاعبين موجودة (كان 404 مقابل 403 يكشفها).

8. الوقت والمناطق الزمنية

أصعب نقطة منطقية في المشروع: «ما هو اليوم؟». المستخدم في دمشق (UTC+3) وقاعدة البيانات تخزّن بـ UTC.

- عند الحفظ: `consumed_at` يُستقبل مع إزاحته (`+03:00`) ويُحوَّل إلى UTC قبل التخزين.
- عند الاستعلام عن يوم `2026-11-07` : نبني بداية اليوم `00:00` بتوقيت `Asia/Damascus` (المقروء من `config/meal_tracking.php`) وبداية اليوم التالي، ثم نحوّلهما إلى UTC، ونبحث بـ `consumed_at >= start AND consumed_at < end`.
- مثال: وجبة الساعة 00:30 بتوقيت دمشق تُخزَّن UTC 21:30 لليوم السابق، لكنها تظهر في يومها الصحيح عند اللاعب.
- الشرط `before_or_equal:now` يمنع تسجيل وجبة في المستقبل.

9. الأمان وعزل البيانات

الخطر	كيف عالجناه
وصول بدون تسجيل دخول	كل المسارات خلف `auth:sanctum` ؛ JSON 401.
لاعب يقرأ/يعدّل سجل لاعب آخر (IDOR)	كل استعلام يبدأ من `$athlete->mealLogs()` أو `$nutritionist->nutritionPlans()` ؛ سجل غيره = 404.
انتحال صاحب السجل (Mass Assignment)	`athlete_id` و `user_id` محظوران (`prohibited`) في الجسم، و `athlete_id` ليس ضمن `$fillable` ؛ يُحدَّد من المستخدم المسجَّل فقط.
ربط سجل بوجبة مخططة لا تخصه	قاعدة `exists:meals` مقيَّدة بوجبات خطط هذا اللاعب فقط.
تصعيد الصلاحيات	`role` و `is_active` ليسا في `$fillable` للمستخدم (لا يمكن تعيينهما بتمرير مصفوفة). والصلاحية تُفحَص في كل FormRequest/Controller.

تخمين كلمات المرور	Throttle 5/دقيقة على login، وتجزئة bcrypt، ونفس الرسالة لكل الأخطاء.
تسريب تواجد الحسابات/اللاعبين	رسالة login موحّدة، و 403 موحّد في عرض الأخصائي.
تسرّب توكن	صلاحية 8 ساعات، يُخزَّن مُجزَّأً، ويُلغى عند logout.
تشغيل Seeder التجريبي في الإنتاج	الـ Seeder يرمي استثناءً خارج بيئتي local/testing.

10. ماذا وجدنا وماذا أصلحنا

بدأنا بفحص كامل للمشروع. ما قيل لك سابقاً (أن MealLog Model و Meal Tracking API و AuthController غير منفّذة) لم يكن دقيقاً: كلها كانت موجودة. لكن الخدمة **لم تكن تعمل فعلياً** بسبب الأخطاء التالية:

#	المشكلة	الإصلاح
1	**حرجة:** `Athlete` ليس له علاقة `mealLogs()` بينما الـ Controller يستدعيها في كل العمليات ← كل واجهات السجلات تنهار بخطأ 500.	إضافة `mealLogs(): HasMany` في `Athlete.php`.
2	**حرجة:** في `SaveMealLogRequest` كانت `'required'` مكرّرة في الحقول، فيستحيل `PATCH` الجزئي (يطلب كل الحقول دائماً).	إبقاء `$presence` فقط: required في POST و sometimes في PATCH.
3	طلب بدون توكن وبدون `Accept` يعطي 500 (`Route [login] not defined`) بدل 401.	تعديل `redirectGuestsTo(null)` : `bootstrap/app.php` و `shouldRenderJsonWhen` لـ `api/*`.
4	**خطر:** `phpunit.xml` يستخدم قاعدة MySQL الحقيقية، وأي اختبار بـ `RefreshDatabase` سيمسحها.	تفعيل SQLite في الذاكرة للاختبارات.
5	الـ Seeder لا ينشئ سجلات وجبات ولا يُستدعى من `DatabaseSeeder`.	إضافة وجبات مخططة وثلاثة سجلات تجريبية (idempotent)، واستدعاؤه في local/testing.
6	لا اختبارات (ملف placeholder فقط).	39 اختباراً.
7	لا يوجد API للأخصائي لإنشاء الخطط والوجبات.	Resources + Requests + `NutritionistController` + مسارات (القسم 7).
8	الأرقام تُرجَع نصاً (`"450.00"`) في السجلات وأرقاماً في الملخص؛ بلا ترقيم صفحات.	Resources موحّدة (`calories` رقم) وترقيم صفحات.

9	`.env.example` يخالف `.env` (SQLite مقابل MySQL)، والـ README هو الافتراضي.	مزامنة الملف وكتابة README بخطوات التشغيل والمسارات.
10	توقّف التثبيت: إضافة `zip` في PHP (XAMPP) كانت معطّلة فكان Composer ينزّل المكتبات بـ `git clone` كاملاً (نحو 3.4 GB).	تفعيل `extension=zip` في `php.ini` وتنظيف الكاش.

قرار متعمَّد: لم نضف `role` و `is_active` إلى `$fillable` في `User` (كان بنداً اختيارياً)، لأن ذلك يفتح ثغرة تصعيد صلاحيات إن استُخدم `User::create($request->all())` يوماً. الـ Seeder يستخدم `forceFill` عن قصد.

11. الاختبارات و Postman

الاختبارات الآلية: 39 اختباراً (161 تأكيداً) — كلها ناجحة

تعمل بـ `php artisan test` على SQLite داخل الذاكرة (لا تمسّ MySQL). كل اختبار يبدأ من قاعدة نظيفة يملؤها الـ Seeder.

الملف	ماذا يغطي
`MealTrackingTest` (22)	login نجاح/فشل/حساب معطّل، logout يلغي التوكن، 401 JSON بدون توكن، إنشاء سجل، رفض `athlete_id` ، رفض تاريخ مستقبلي، نوع وجبة خاطئ، منع الأخصائي من التسجيل، فلترة اليوم، عزل اللاعبين، PATCH جزئي، حذف، ملخص (متبقي/زيادة)، planned-meals، وصول الأخصائي المسموح والممنوع.
`NutritionistPlanTest` (15)	إنشاء خطة، منع اللاعب من إدارة الخطط، أخطاء التحقق، منع تداخل الخطط، PATCH جزئي، منع تغيير اللاعب وكسر التواريخ، التملّك (404 لأخصائي غريب)، حذف خطة يحذف وجباتها ويبقي السجلات، CRUD وجبات، قائمة اللاعبين، ظهور وجبة الأخصائي عند اللاعب، 403 موحّد، ترقيم الصفحات وأرقام السعرات.
اختباران افتراضيان	Example tests من Laravel.

إضافةً إلى ذلك شغّلنا الخدمة فعلياً (`php -S`) على قاعدة SQLite مؤقتة وجرّبنا login ثم `meal-logs` و `summary` ورد 401 بلا توكن، وكانت النتائج صحيحة (الأرقام الظاهرة في هذا المستند منها).

مجموعة Postman

الملفان في مجلد `postman/` : `FitSync_MealTracking.postman_collection.json` و `FitSync_Local.postman_environment.json` . تحتوي 39 طلباً في 7 مجلدات تُشغَّل بالترتيب (Collection Runner):

1. **Auth:** دخول لاعب، لاعب آخر، أخصائي (يحفظ التوكنات تلقائياً في متغيرات) + دخول خاطئ.
2. **Meal Logs:** إنشاء، عرض باليوم، تعديل جزئي.
3. **Summary & Plan:** الملخص اليومي والوجبات المخططة.
4. **Nutritionist:** عرض سجلات لاعب.
5. **Nutritionist - Plans & Meals:** إنشاء خطة، رفض التداخل، تعديل، إضافة وتعديل وحذف وجبة مخططة، حذف الخطة.
6. **Negative tests:** بدون توكن، تاريخ ناقص، `athlete_id` في الجسم، تاريخ مستقبلي، نوع خاطئ، سعرات سالبة، لاعب يعدّل سجل غيره، أخصائي يسجّل وجبة.
7. **Cleanup & Logout:** حذف السجل، logout، والتأكد أن التوكن لم يعد يعمل.

كل طلب يحتوي سكربت `pm.test` يفحص كود الرد وحقوله. المتغيرات (`consumedAt`, `today`, ...) تُحسب تلقائياً بتوقيت دمشق.

> ملاحظة أمانة: ملف Postman تحقّقنا من صحة بنيته ومن أن مساراته وأجسامه مطابقة للـ API المختبَر، لكننا لم نشغّله داخل برنامج Postman نفسه. شغّله مرة قبل المناقشة للتأكد.

12. تشغيل المشروع وسيناريو العرض

التشغيل

```
1) شغّل MySQL من XAMPP  (قاعدة fitsync)
2) composer install
3) php artisan migrate:fresh --seed     # تحذير: يمسح الجداول ويعيد بناءها
4) php artisan serve                    # http://127.0.0.1:8000
5) php artisan test                     # اختياري: 39 اختباراً
```

الحسابات التجريبية (كلمة المرور `DemoPassword!123`): `athlete@example.test`، `other@example.test`، `nutritionist@example.test`.

سيناريو عرض مقترح (5 دقائق)

1. **الأخصائي يدخل** (Postman: Login - Nutritionist) ← يعرض لاعبيه ← ينشئ خطة للاعب بهدف 1800 سعرة ← يضيف وجبة مخططة.
2. **يُظهر الحماية:** إنشاء خطة تتداخل مع خطة فعّالة ← 422.
3. **اللاعب يدخل** ← يستعرض `planned-meals` فيرى وجبة الأخصائي.
4. **اللاعب يسجّل وجبة** ← يعرض اليوم ← يعدّل السعرات بـ PATCH ← يشاهد `summary` (المسجَّل / الهدف / المتبقي).
5. **العزل:** لاعب آخر يحاول تعديل هذا السجل ← 404؛ أخصائي غير مرتبط يحاول عرضه ← 403.
6. **الأخصائي يتابع** سجلات لاعبه ليوم معيّن.
7. في النهاية يُشغَّل `php artisan test` أمامها: 39 ناجحاً.

13. أسئلة متوقعة وأجوبتها

لماذا Sanctum وليس JWT أو Session؟

Sanctum مدمج في Laravel ومناسب لـ API يستهلكها تطبيق موبايل أو SPA: توكنات بسيطة تُخزَّن مجزّأة في القاعدة، تُلغى فورياً (logout)، وتحمل صلاحية ووقت انتهاء. JWT لا يمكن إلغاؤه قبل انتهاء صلاحيته بدون قائمة سوداء، والـ Session تحتاج كوكيز (مناسبة للمتصفح لا للـ API).

كيف تضمنون أن لاعباً لا يرى بيانات لاعب آخر؟

لا نأخذ `athlete_id` من العميل أبداً. نحدّده من المستخدم المسجَّل، وكل استعلام يبدأ من `$athlete->mealLogs()`. لذلك سجل غيره غير موجود من وجهة نظره (404). ولدينا اختبارات تثبت ذلك.

لماذا تخزنون الوقت بـ UTC؟ وكيف تحسبون «اليوم»؟

التخزين بـ UTC معيار يتفادى مشاكل التوقيت الصيفي والسفر. «اليوم» يُحسب بتوقيت دمشق: نحوّل حدّي اليوم (00:00 – 00:00) إلى UTC ونبحث بينهما، فتظهر كل وجبة في يومها المحلي الصحيح.

لماذا ترفضون خطتين فعّالتين متداخلتين؟

لأن الملخص اليومي يقارن بهدف واحد. لو وُجد هدفان لليوم نفسه فلا معنى للرقم «المتبقي». نمنعها عند الإنشاء/التعديل (422)، وكخط دفاع ثانٍ يردّ الملخص 409 إذا وُجد تداخل بطريقة أخرى.

ما الفرق بين Meal و MealLog؟

`Meal` وجبة **مخططة** يضعها الأخصائي ضمن خطة. `MealLog` ما أكله اللاعب **فعلاً**. يمكن ربطهما اختيارياً (`meal_id`) لمقارنة المخطط بالفعلي، وإذا حُذفت الوجبة المخططة يبقى السجل.

ماذا يحدث عند حذف خطة؟

تُحذف وجباتها المخططة وخطتها داخل `DB::transaction` (كلها أو لا شيء)، وسجلات اللاعب لا تُحذف: يصبح `meal_id` فيها `NULL`. مثبَت باختبار.

كيف تتعاملون مع الأخطاء؟

كل أخطاء JSON `api/*` بكود مناسب: 401 غير مسجَّل، 403 لا صلاحية، 404 غير موجود/ليس لك، 409 تعارض خطط، 422 بيانات خاطئة مع تفصيل كل حقل، 429 طلبات كثيرة على login.

لماذا استخدمتم FormRequest و Resource؟

فصل المسؤوليات: FormRequest للتحقق والصلاحية (يُعاد استخدامه ويُختبر بسهولة)، Resource لشكل الرد المستقل عن شكل الجدول، فنستطيع تغيير القاعدة دون كسر العميل.

كيف تثبتون أن الكود يعمل؟

39 اختبار آلي (`php artisan test`) تغطي المسارات الناجحة والفاشلة والأمان، إضافةً إلى مجموعة Postman وتجربة تشغيل حقيقية للسيرفر.

ما الأخطاء التي وجدتموها في الكود الأصلي؟

أهمها: علاقة `mealLogs()` الناقصة في `Athlete` (تُسقط كل الواجهات)، وتكرار `required` الذي يمنع التعديل الجزئي، وخطأ 500 بدل 401، وإعداد الاختبارات الذي كان سيمسح قاعدة البيانات الحقيقية. التفاصيل في القسم 10.

14. حدود الخدمة وتطويرات مستقبلية

من الأفضل أن تذكرها بنفسك قبل أن تُسأل عنها:

- **لا يوجد تسجيل حساب (Register)** ولا إدارة مستخدمين: الحسابات تُنشأ عبر الـ Seeder. لوحة إدارة أو endpoint للتسجيل خطوة قادمة.
- **الأخصائي يستطيع إنشاء خطة لأي لاعب** بدون موافقة اللاعب (طلب/قبول ارتباط). هذا قرار مبسّط في هذه المرحلة.
- **الدور نص** (`athlete` / `nutritionist`) وليس Enum في القاعدة، والتحقق في الكود.
- رؤية الأخصائي لسجلات لاعبه محصورة بخطة فعّالة اليوم؛ لا يستطيع مراجعة سجلات قديمة بعد انتهاء الخطة.
- لا يوجد حذف ناعم (soft delete) ولا سجل تدقيق (audit log).
- السعرات فقط — لا ماكرو (بروتين/كارب/دهون) ولا قاعدة بيانات أطعمة.
- Throttle مطبَّق على login فقط.
- تطويرات مقترحة: Macros، قوالب وجبات، إشعارات تجاوز الهدف، رسوم أسبوعية، تسجيل مستخدمين، وربط اللاعب بالأخصائي بطلب وقبول.

— انتهى —